

WER Is Not Enough: How I Benchmarked and Fine-Tuned ASR for Indian Banking

11 min read · Just now



Nakshatra Kanchan



Listen



Share



More

Three weeks on fine-tuning, multiple models, one very specific problem: making an ASR pipeline actually work for Indian banking calls.

A customer calls a debt collection bot and says: *“Mera account number hai 9876543210, aur outstanding amount kitna hai?”*

A model with 8% overall WER on a standard Hindi benchmark might transcribe “9876543210” as “9876543110” — a single digit off. The downstream slot-filler extracts the wrong account. The bot queries the wrong record. The call fails. The model’s report card still says 8%.

This is the problem with WER as a single-number metric for banking ASR. It counts a garbled digit the same as a garbled article. It treats a missed “Aadhaar” the same as a missed filler word. In a domain where a wrong digit in a loan amount or an OTP can have real consequences, optimizing for WER alone will get you a system that looks great on paper and fails in production.

Over the past three weeks, I’ve been building the ASR evaluation and fine-tuning pipeline for krim.ai’s banking voice bot — which handles debt collection, loan origination, and call center support for autonomous banking. This post covers what we found: why WER isn’t the right metric, which models actually hold up on Indian



banking speech, what fine-tuning buys you, and what the production architecture should look like for 500+ concurrent calls.

1. Why WER Isn't Enough for Indian Banking

The standard pitch for WER is that it's model-agnostic and easy to compute. Run `jiwer.wer(reference, hypothesis)`, get a number, compare. The problem is what that number hides.

In Indian banking, the failure modes are specific and asymmetric. Three categories of errors matter more than all others combined.

Numeric precision. Account numbers, OTPs, phone numbers, loan amounts — a single transposed or substituted digit is a catastrophic failure, not a minor inconvenience. Standard WER treats “9876543210” → “9876543110” as one substitution error in a ten-word sentence, giving you 10% WER for an input that will break your system. We measure this separately as **Number Error Rate (NER)**: WER computed only over numeric tokens extracted from the transcript.

Domain entity accuracy. Named entities like “CIBIL”, “Aadhaar”, “NACH”, “KYC”, and bank names like “HDFC” or “SBI” must come out exactly right to trigger the correct intent. A model that transcribes “CIBIL score” as “civil score” has a low character-level error but has broken your intent classifier. We track these separately as **Entity Accuracy**: the fraction of core banking entities transcribed correctly.

Code-switching. The majority of customers calling an Indian banking bot speak in Hinglish — a blend of Hindi and English that switches scripts mid-sentence. “*EMI payment date kya hai?*” is a completely natural sentence. Models trained on either pure Hindi or pure English handle this poorly for different reasons, and a composite WER score hides whether the model is failing on the Hindi segments, the English segments, or the transitions.

The full metric suite we ran: WER and CER for general transcription quality, NER for numeric precision, Entity Accuracy for banking terminology, and RTF (processing time / audio duration) for latency. A model that gets 8% WER but 25% NER on banking audio is dangerous in production. These numbers don't move together.

2. The Dataset Problem

There is no public dataset that covers banking Hinglish. This is the first problem you hit when you try to do this work properly.

What exists, and what each dataset is actually good for:

Kathbath is the largest Hindi ASR dataset (1750 hours) and serves as the standard general-Hindi benchmark. It has accent metadata and a well-defined validation split. This is your baseline evaluation set for general Hindi quality. It tells you nothing about banking speech.

MUCS 2021 (Hindi-English Finance track) is 90 hours of lecture-style speech covering financial topics in Hindi and English. It is not conversational and not code-switched in the Hinglish sense, but it gets you domain vocabulary. The data is distributed in Kaldi format — `wav.scp`, `text`, `segments` — with multi-minute recordings that need to be sliced into utterances before training. We wrote a parallelized slicer using `soundfile` and `ProcessPoolExecutor` to cut and save thousands of utterance-level `.wav` chunks and emit a JSON Lines manifest. I'll come back to why this matters when I describe the fine-tuning configs.

RESPIN-S1.0 covers agriculture and finance domains in regional Indian accents. It's structured like MUCS: Kaldi format, needs metadata scanning, benefits from parallelization. On Param Rudra's network filesystem (NFS/GPFS), sequential metadata scans across large directories are slow enough to matter — we parallelized the RESPIN scanner with 32 threads and dropped metadata scanning time from over 15 minutes to under 20 seconds.

Project Vaani (IISc) is ~300 GB of transcribed Hindi speech from across the Hindi belt. It's massive, and the majority is clean read speech rather than conversational. Its value is preventing domain forgetting when fine-tuning on narrow banking data.

LAHAJA is the accent evaluation dataset that most people doing this work miss. 6,152 conversational samples from 132 speakers across 83 districts, covering 19 native language families. This is what you use to evaluate whether your model degrades on south Indian Hindi speakers, east Indian speakers, or Punjab-Haryana speakers. We'll get to those numbers.

None of this covers actual banking call center speech. For that, we generated a synthetic test set: 100 Hinglish banking queries generated via Gemini (account balance inquiries, EMI questions, OTP flows, loan status checks), converted to audio via TTS, and annotated with reference transcripts and entity labels. This is `synthetic_100` — the primary test set for banking-domain evaluation. It's not a substitute for real call recordings, but it's a reproducible baseline that captures the core challenges: code-switching, numeric precision, and domain vocabulary.

3. Zero-Shot Benchmark: Who Actually Works Out of the Box

We ran every serious model on two test sets: Kathbath (3,151 samples, general Hindi) and `synthetic_100` (100 samples, banking Hinglish). The results do not look the same.

General Hindi (Kathbath)

Model	Architecture	WER (%)	CER (%)	RTF
IndicWav2Vec-Hindi	Wav2Vec 2.0 (Mono)	11.64%	3.30%	0.09
Nemotron-3.5-ASR	Transducer (RNNT)	13.00%	4.44%	0.174
Conformer-CTC Large	Conformer CTC	13.26%	3.99%	0.013
Voxtral-Mini-3B	Encoder-Decoder (LLM)	17.66%	6.72%	0.54
Whisper-large-v3-turbo	Encoder-Decoder	32.01%	10.37%	0.084
Whisper-medium-hi	Encoder-Decoder	41.64%	15.85%	0.96

Hinglish Banking (Synthetic 100)

Model	WER (%)	Entity Accuracy (%)	RTF
Voxtral-Mini-3B	46.01%	70.5%	0.54
Whisper-large-v3	46.32%	67.0%	1.49
Whisper-large-v3-turbo	55.52%	62.0%	0.50
Nemotron-3.5-ASR	67.81%	—	0.174
Conformer-CTC Large	73.61%	—	0.013
IndicWav2Vec-Hindi	75.49%	29.0%	0.09
Whisper-medium-hi	179.17%	74.0%	0.96

The headline number is IndicWav2Vec: **11.64% WER on Kathbath, 75.49% on banking Hinglish**. This is not a minor degradation. It's complete collapse, and the reason is structural: IndicWav2Vec is a character-based CTC model trained exclusively on Devanagari script. Its vocabulary contains no Latin characters. When a caller says “credit card block ho gaya hai”, the model cannot output the Latin letters for “credit card”. It either maps to phonetically similar Devanagari sequences or drops the tokens entirely. Either way: substitution error, deletion error, WER spike. This is not fixable with more Hindi training data. The vocabulary constraint is baked into the tokenizer.

Whisper-medium's 179.17% WER deserves its own paragraph. A WER above 100% means the model is inserting more words than exist in the reference — infinite repetition loops. The autoregressive decoder, confronted with short noisy Hinglish utterances, enters a feedback cycle where it repeatedly generates the same tokens. We saw sequences like “hai hai hai hai...” filling the hypothesis for a five-word reference. This is Whisper's known repetition loop problem on non-standard speech.

Voxtral-Mini-3B wins zero-shot on banking at **46.01% WER** with **70.5% entity accuracy**. It handles code-switching because it's a multilingual LLM-based ASR model — it doesn't have a script constraint. But its RTF of 0.54 on an A100 means it can't handle 500 concurrent calls on a single node. Running an autoregressive LLM over hundreds of simultaneous audio streams is architecturally the wrong approach for a live bot.

The Pareto picture: Conformer-CTC Large at **RTF 0.013** is in a different class for latency. A mean chunk latency of **49ms** on an A100 is effectively instantaneous. Its zero-shot Hinglish WER (73.61%) is not great, but it’s a bilingual model with a combined Devanagari + Latin vocabulary — the script barrier doesn’t apply. Nemotron-3.5-ASR at **0.174 RTF** is the streaming champion: it processes audio chunk-by-chunk as the user speaks, so the user-perceived final delay is near zero in production even if the offline RTF looks worse than Conformer-CTC.

Excluded Models (language mismatch — no Hindi support)

Model	Backend	Reason
Parakeet-TDT-0.6B	NeMo	English-only — romanized Hindi audio
Canary-1B-Flash	NeMo	EN/DE/FR/ES only — no Hindi support
Streaming Zipformer	Sherpa-ONNX	English-only (LibriSpeech) — output nonsense on Hindi
IndicConformer-Hindi	NeMo	Requires AI4Bharat's NeMo fork (<code>nemo-v2</code>), incompatible with standard NeMo

4. Fine-Tuning: What Actually Worked

We designed three training configs to study the effect of dataset composition:



Config B — 639 Gemini-generated synthetic Hinglish banking queries. Purely synthetic, smallest dataset, fast iteration.

Config C — Config B + MUCS Hindi-English Finance (90h). Adds real lecture-style speech with domain vocabulary.

Config D — Config C + Project Vaani (Hindi belt subset) + RESPIN-S1.0. The large-scale native speech addition is specifically to prevent catastrophic forgetting.

WER Comparison: Baseline vs. Fine-Tuned

WER Comparison: Baseline vs. Fine-Tuned

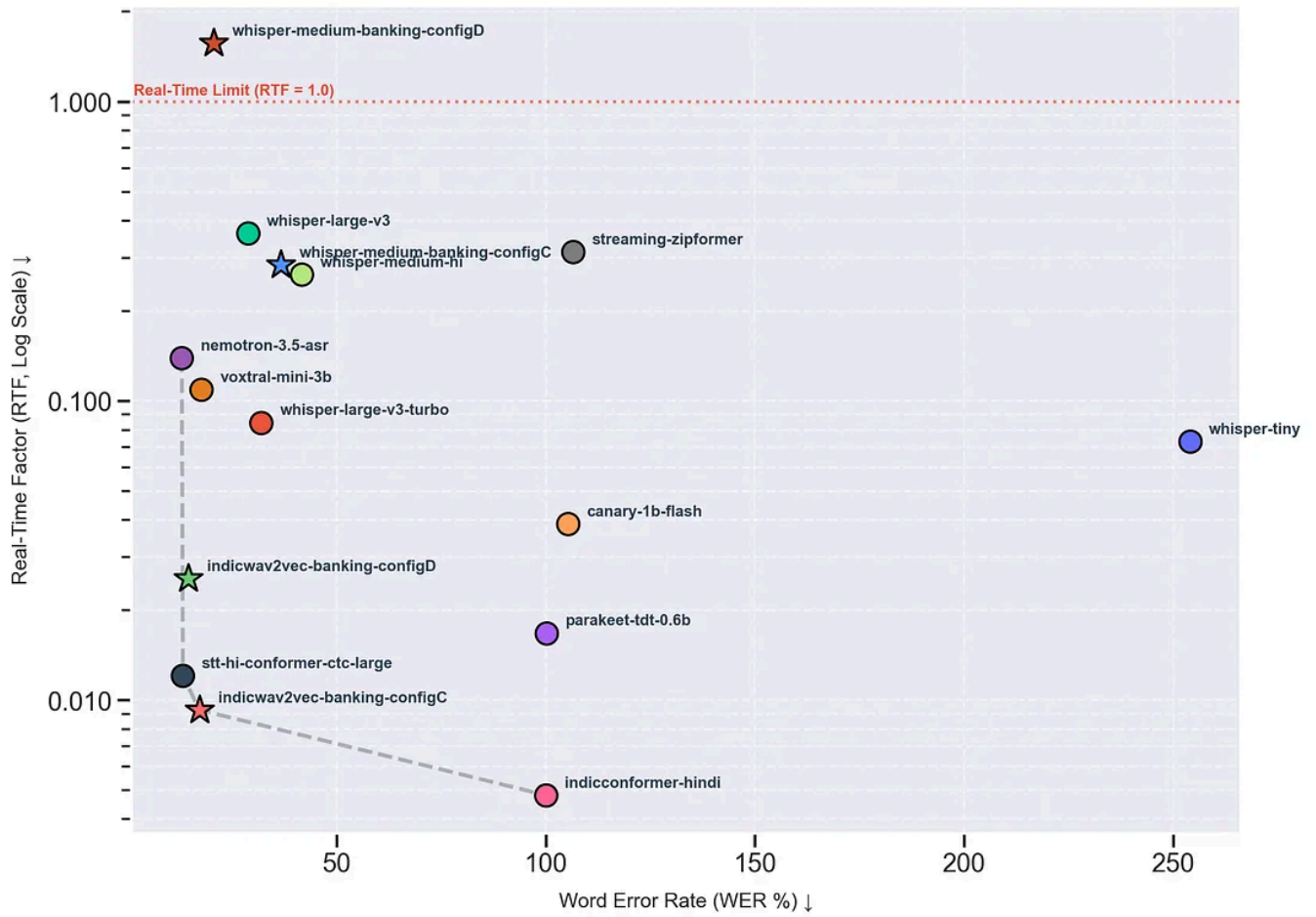
Model & Config	Hinglish Banking (synthetic_100)	General Hindi (Kathbath)
IndicWav2Vec-Hindi (Baseline)	75.49%	11.64%
IndicWav2Vec-banking-configC	78.95%	17.20%
IndicWav2Vec-banking-configD	73.35%	14.52%
Whisper-medium-hi (Baseline)	179.17%	41.64%
Whisper-medium-banking-configC	37.74%	36.61%
Whisper-medium-banking-configD	48.73%	20.57%

The Whisper result is the headline: Config C takes Hinglish WER from **179.17% down to 37.74%** — a **141.43% absolute reduction**. Domain adaptation on code-switched banking data eliminated the hallucination loops almost entirely (more on that in the error analysis section) and taught the model the vocabulary distribution of actual banking queries.

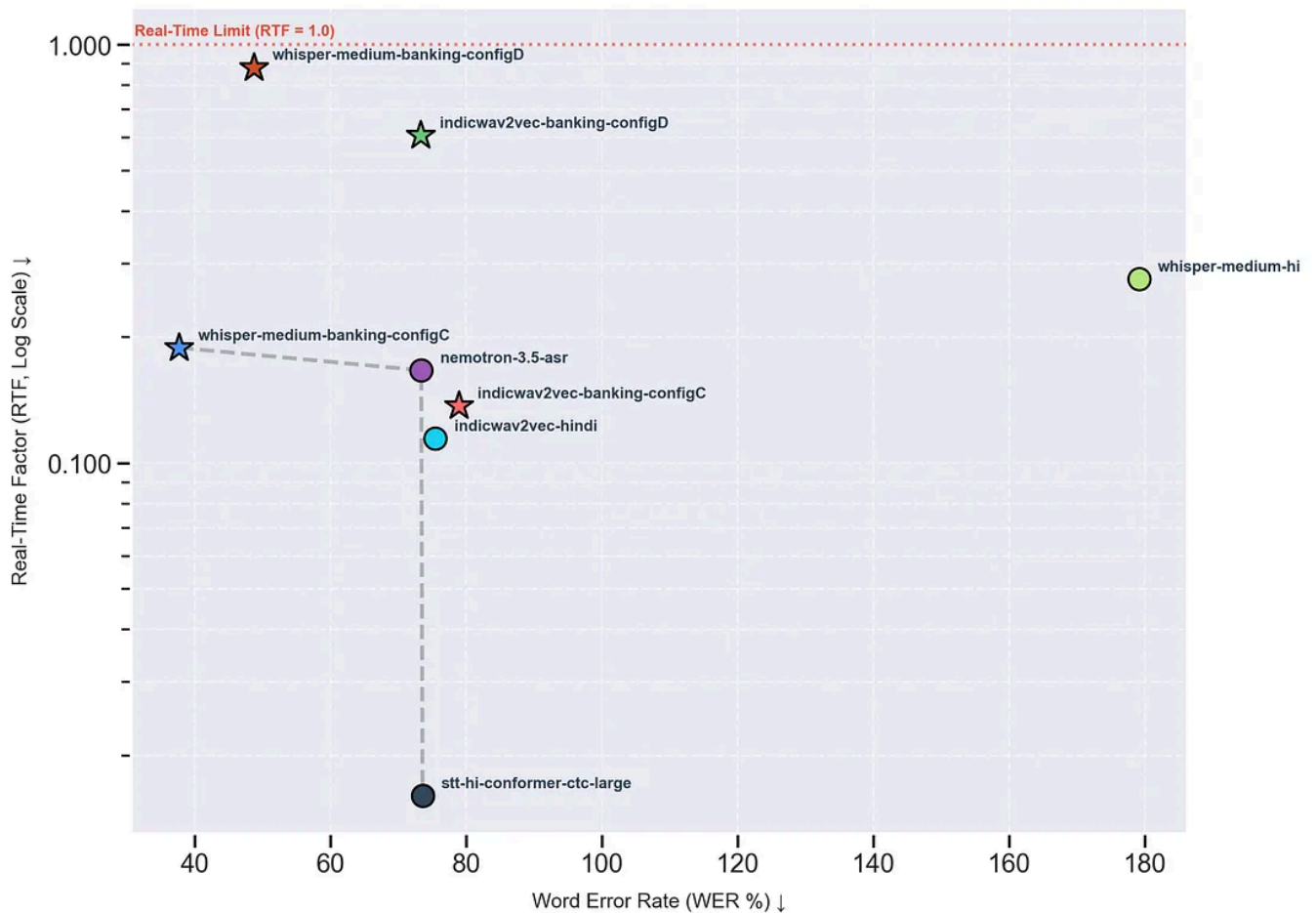
The IndicWav2Vec result is the lesson about structural limits: fine-tuning on Config C actually made Hinglish WER slightly *worse* (75.49% → 78.95%). Config D partially recovered it (73.35%), but the gap between Whisper-medium fine-tuned and IndicWav2Vec fine-tuned is 37.74% vs 73.35%. That gap won't close with more training data. It's the vocabulary constraint.

The Config D story is about forgetting. Whisper fine-tuned on Config C had general Hindi WER of 36.61% — an improvement over baseline (41.64%), but not spectacular. Config D, which added Vaani and RESPIN to the training mix, brought it to **20.57%** — a 16.04% absolute improvement over Config C. Large-scale native speech corpora are not optional if you want both domain performance and general Hindi robustness. The intuition is that Vaani gives the model enough clean Hindi speech to retain its prior on phonetics even as the domain vocabulary shifts toward banking Hinglish.

ASR Quality-Latency Pareto Frontier (General Hindi (Kathbath))



ASR Quality-Latency Pareto Frontier (Banking Hinglish (Synthetic 100))



5. What the Errors Actually Are

Reporting WER numbers is one thing. Knowing *which kinds* of errors your model is making tells you whether a problem is fixable with more data or is architectural.

We wrote an alignment classifier that maps word-level mismatches against `synthetic_100` into five categories:

Error Category	IndicWav2Vec (Config C)	Whisper (Config C)	Whisper-medium Baseline
Entity Deletions	47.42%	23.55%	5.53%
Number Substitutions	29.08%	43.31%	9.74%
Hindi/Latin Script Mismatch	17.39%	28.78%	3.41%
Hallucinations (Insertions)	0.14%	0.58%	78.45%
Other Substitutions/Deletions	5.97%	3.78%	2.87%

The baseline Whisper number tells the full story: **78.45% of its errors are hallucinations** — infinite repetition insertions. Fine-tuning on Config C drops this to **0.58%**. The domain distribution of training data was enough to stabilize the decoder.

After fine-tuning, Whisper’s biggest remaining problem is numbers — **43.31%** of errors are digit substitutions. This is not surprising: numeric sequences in Hinglish are spoken in a mix of Hindi number words (“do hazaar”) and digit-by-digit English (“four four three five”), and the model has to handle both. The IndicWav2Vec failure mode is different: **47.42% entity deletions**, meaning the model is simply dropping banking entities it can’t represent in Devanagari. “CIBIL score” → nothing. The script barrier shows up at the error level.

The combined script mismatch category (Hindi/Latin) drives **21.60%** of IndicWav2Vec Config C errors. This is structural: any banking term written in Latin script is outside the model’s generation vocabulary. Fine-tuning teaches it to recognize those audio patterns, but it still can’t emit the correct Latin characters.

6. Accent Evaluation: LAHAJA

The standard Indian ASR benchmark won't tell you whether your model works for a Tamil speaker who learned Hindi as a second language or a Bengali speaker in Kolkata's call center workforce. LAHAJA covers this: 6,152 samples from 132 speakers across 83 Indian districts, spanning 19 native language families. Accent groups are mapped to five regions: Punjab/Haryana, Hindi Belt, South India, East India, and West India.

Accent Group	IndicWav2Vec (Baseline)	Conformer- CTC (Baseline)	Nemotron- 3.5 (Baseline)	IndicWav2Vec (Config D)	Whisper (Config D)
punjab_haryana	28.16%	19.74%	21.89%	26.67%	30.73%
hindi_belt	26.48%	18.37%	20.19%	27.20%	29.44%
south_india	35.11%	25.46%	26.25%	36.09%	36.38%
east_india	32.81%	26.15%	26.13%	34.79%	37.71%
west_india	30.41%	20.36%	20.75%	32.13%	34.74%
Overall Mean	33.22%	24.58%	25.09%	34.99%	38.27%

Conformer-CTC Large wins zero-shot dialect robustness with **24.58% overall WER**. Nemotron is close at **25.09%**. IndicWav2Vec at **33.22%** trails both. South India is the hardest accent group across every model — the phonetic distance from Hindi L1 speakers is the largest, and that shows up consistently. No model closes this gap cleanly; the best anyone does on south India is Conformer's 25.46%.

The fine-tuned models don't help on accent robustness. Whisper-banking-configD achieves **38.27% overall** on LAHAJA — worse than the zero-shot Conformer or Nemotron baselines. Fine-tuning on banking data improves banking domain accuracy but doesn't transfer to accent diversity. These are separate problems.

7. Production Recommendation

Three deployment tiers, based on use case:

Live streaming at 500+ concurrent calls → Conformer-CTC Large on Triton with TensorRT.

RTF of 0.013 means 49ms mean chunk latency on an A100. Non-autoregressive CTC inference doesn't have a sequential decode step, so it scales by batching audio chunks across GPU tensor cores. Served via NVIDIA Triton Inference Server with TensorRT-ASR, a single A100 can handle hundreds of simultaneous audio streams with millisecond per-chunk latency. This is the cost-efficient, scalable path. The tradeoff is higher zero-shot Hinglish WER (73.61%) — but in a production pipeline where you have some LLM-based error recovery downstream, the latency properties are the dominant consideration. You can't serve 500 concurrent calls with Voxtral's autoregressive decoder.

Interactive voice assistant with real-time response → Nemotron-3.5-ASR-Streaming-0.6B.

The key distinction between Nemotron and Conformer-CTC is that Nemotron is a native streaming model: it emits partial transcripts as audio arrives, chunk-by-chunk, while the user is still speaking. The user-perceived final latency is effectively zero because transcription and speech happen in parallel. Its 0.174 offline RTF is better than Whisper but looks worse than Conformer-CTC — that comparison is misleading because Conformer's 0.013 is on batch inference, not streaming. Set `target_lang="auto"` so the model switches between Latin and Devanagari dynamically.

Batch/offline analytics → Fine-Tuned Whisper-Medium (Config C).

If you're doing post-call analysis, quality scoring, or building training data from call recordings, latency doesn't matter and accuracy does. Whisper-medium Config C at 37.74% Hinglish WER is the best accuracy number in this evaluation. It's also the only model that doesn't hit the script barrier — it'll handle arbitrary Hindi-English mixing in a single output stream. The hallucination problem is gone after fine-tuning; it's a stable model for offline use.

Don't deploy IndicWav2Vec for banking, full stop. The 11.64% general Hindi WER is impressive and will look good in a benchmark comparison. But on actual banking calls, you get 73–79% WER and 47% entity deletions. The code-switching failure is

not a fine-tuning problem — it's a vocabulary constraint. No amount of training data will let a Devanagari-only tokenizer output Latin characters. For banking, you need a bilingual vocabulary.

- Asr
- Voice Ai
- Fine Tuning
- Speech Recognition
- Indian Languages



Edit profile

Written by Nakshatra Kanchan

0 followers · 1 following

No responses yet



Nakshatra Kanchan he/him

What are your thoughts?

More from Nakshatra Kanchan